

PIFF

Portable Indoor Feedback

Full Project Documentation

Student: Leandro Cerami

Class: REPIF1 | 1TPIFI2

School: Lycée Privé Emile Metz

Year: 2025 – 2026

Table of Contents

Table of Contents	2
L1.1 – Operating System Installation	6
L1.2 – Hardware Assembly	6
L1.3 – Initial Configuration	6
Display Orientation	6
Enabling Interfaces	7
Remote Access Setup	7
L1.4 – System Updates & Docker	7
L1.5 – Folder Structure & Apache	7
L1.6 – File Deployment & Configuration	7
L1.7 – Docker Container & Autostart	8
L2.1 – Data Modelling	9
L2.2 – Database Schema	9
L2.3 – MariaDB Installation	9
PCB Assembly Overview	10
Step 1 – Component Inventory	10
Step 2 – Component Placement	10
Step 3 – Soldering	10
Step 4 – Final Inspection	10
Overview	11
PVC Board 1 (Front/Display Support)	11
PVC Board 2 (Rear/Structural Board)	11
Introduction	12
Materials & Tools	12
Procedure	12
Step 1 – Cable Preparation	12
Step 2 – Wire Identification	12
Step 3 – Wire Stripping	12
Step 4 – Soldering	12
Step 5 – Insulation	13
Step 6 – Final Assembly	13
Safety Measures	13
Conclusion	13
Development Overview	14
Step 1 – Preparing and Soldering the ADC Module	14
Step 2 – Gas Sensor Wiring & PCB Integration	14
Step 3 – Power Distribution & Signal Routing	14
Step 4 – Connecting the PCB to the Raspberry Pi	14

Step 5 – Mechanical Assembly Inside Enclosure.....	15
Step 6 – Software Configuration & Testing	15
Conclusion	15
Introduction	16
Materials List	16
Assembly Process.....	16
3.1 – Component Check	16
3.2 – Preparing the Wooden Housing	16
3.3 – Installing the Push Button	17
3.4 – LED Openings.....	17
3.5 – Mounting the Sensors	17
3.6 – Installing Raspberry Pi & Display	17
3.7 – Connecting All Components.....	17
3.8 – Final Assembly.....	17
System Function.....	17
Problems & Solutions	17
Objective	19
System Preparation.....	19
Installing Apache Web Server	19
Installing MariaDB	19
Securing the Database	19
Installing PHP.....	19
Installing phpMyAdmin	19
Configure Apache	20
Creating the PIF Database	20
Deploying the Website.....	20
Introduction	21
Technologies Used.....	21
User Management.....	21
Stations Management	21
Measurements Visualisation.....	21
Collections.....	22
Friends & Sharing System.....	22
Administrator Area.....	22
UI & Design	22
Conclusion	22
Introduction	23
Requirements.....	23
Network Configuration	23

Station Configuration	23
PHP Data Transfer Script (send.php)	24
API Handler	24
Web Form	24
API Reference	24
Testing Results	24
Conclusion	25
Introduction	26
System Architecture	26
Virtual Machine Setup	26
Database Remote Access Configuration	26
MySQL Bind Address	26
Firewall Rule	26
Backup User (Principle of Least Privilege)	27
Backup Implementation	27
Secure Credential Storage	27
Backup Script (/usr/local/bin/backup_database.sh)	27
Automated Scheduling (Cron)	27
FTP Server Configuration	27
Database Recovery Procedure	28
Issues & Resolutions	28
Conclusion	28
Objective	29
Software Stack	29
Complete Setup Procedure	29
1. System Update	29
2. Apache Web Server	29
3. MariaDB Database Server	29
4. PHP Installation	29
5. phpMyAdmin	30
6. Database Creation	30
7. Website Deployment	30
Final Server Summary	30

PROJECT OVERVIEW

Portable Indoor Feedback — PIF

The Portable Indoor Feedback (PIF) project is a comprehensive IoT-based environmental monitoring system developed at Lycée Privé Emile Metz. The system collects real-time environmental data — temperature, humidity, atmospheric pressure, light intensity, and gas concentration — using a Raspberry Pi-based station equipped with dedicated sensors.

The data is transmitted to a central web server, stored in a relational database, and visualised through a PHP web platform. The full solution spans hardware assembly, PCB soldering, sensor integration, server configuration, backup infrastructure, and web development.

Component	Description
Hardware Platform	Raspberry Pi 4 inside a custom wooden/PVC housing
Sensors	Gas (MQ-135 analog), Light (BH1750), Barometric (BME280)
PCB	Custom PIF 25/26 board with ADS1115 16-bit ADC
Web Stack	Apache2 + PHP + MySQL (MariaDB) + Chart.js
Data Transfer	HTTP POST from Raspberry Pi to PHP endpoint (send.php)
Backup	Ubuntu Server VM — automated mysqldump + FTP access
Enclosure	Wooden housing (5-part) + PVC bracket for Raspberry Pi & display

L1 – RASPBERRY PI SETUP

Weeks 46 – 47

L1.1 – Operating System Installation

The first step was to install Raspberry Pi OS onto a microSD card using the official Raspberry Pi Imager tool.

1. Download and install Raspberry Pi Imager from: <https://www.raspberrypi.com/software/>
2. Launch the imager and select the device: Raspberry Pi 4.
3. Select OS: Raspberry Pi OS (64-bit, latest recommended version).
4. Insert the microSD card into the PC via a USB adapter.
5. Click Next and configure the advanced settings.
6. In General Settings: set hostname, username, password, WLAN (school network), and locale.
7. In Services: enable SSH with password authentication.
8. Click Yes to apply settings, then confirm flashing. Back up any existing data first.
9. Remove the microSD card and insert it into the Raspberry Pi.

L1.2 – Hardware Assembly

The following hardware components were assembled to create a touchscreen indoor weather station:

- Raspberry Pi 4 with microSD card
- Power supply
- 7-inch touchscreen display for Raspberry Pi
- Power cable
- Standard-standard ribbon cable
- Standard-mini ribbon cable
- 8 × M2.5 screws

All components were mounted together according to the assembly guide. After mounting, the Raspberry Pi was connected to a power outlet and booted automatically.

L1.3 – Initial Configuration

Display Orientation

The default display resolution is in portrait (vertical) mode. To switch to landscape:

- Click the Raspberry Pi logo → Preferences → Control Center
- Scroll to Screens → touch screen (DSI-1) → Orientation → Right

Enabling Interfaces

The following interfaces were enabled via the Interfaces tab in Raspberry Pi Configuration:

- VNC (Virtual Network Computing) — for remote desktop / screen sharing
- I2C (Inter-Integrated Circuit) — two-wire serial protocol for sensor communication
- SSH — already enabled during OS installation

Remote Access Setup

To connect remotely, the IP address was retrieved using:

```
hostname -I
```

Then RealVNC Viewer was used to connect to the Raspberry Pi from another machine.

L1.4 – System Updates & Docker

After first boot, system updates were applied and Docker was installed:

```
sudo apt update && sudo apt upgrade
# Download and run Docker install script
sudo bash get.docker.sh
sudo systemctl status docker
```

Docker containers create isolated environments similar to virtual machines without running a full OS, enhancing portability and convenience. Docker is used here to host the station's local web interface.

L1.5 – Folder Structure & Apache

Required directories were created with correct permissions:

```
sudo mkdir /opt/station && sudo chmod 755 /opt/station
sudo mkdir /var/station && sudo chmod 755 /var/station
```

Apache2 was installed to serve the station's web interface:

```
sudo apt install apache2
sudo systemctl status apache2
```

L1.6 – File Deployment & Configuration

School-provided firmware, scripts, and website files were transferred from a Windows machine using WinSCP (drag-and-drop via SSH connection):

- Website files → /var/www/html
- Firmware files → /opt/station

The station configuration file was adjusted for the local environment:

```
sudo nano /opt/station/config.toml
```

This file defines: station serial number, I2C addresses, measurement interval, and destination server IP address.

L1.7 – Docker Container & Autostart

A PHP 8.4 Apache Docker container was created to host the station website:

```
docker run -d --name php -p 8080:80 -v /tmp/station:/tmp/station -v /opt/station/www:/var/www/html php:8.4-apache
```

An autostart script was configured at ~/.config/labwc/autostart to launch all services on boot:

```
mkdir -p /tmp/station
sudo python3 /opt/station/measure.py > /tmp/station/result.csv
sudo docker start php
cd /opt/station && source venv/bin/activate
chromium --kiosk http://localhost:8080
sudo chmod +x root/.config/labwc/autostart
```

L2 – DATABASE DESIGN & IMPLEMENTATION

Weeks 48 – 49

L2.1 – Data Modelling

Before implementing the database, two conceptual models were designed:

- CDM (Conceptual Data Model) — high-level representation of entities and relationships without technical detail
- LDM (Logical Data Model) — technical structure with primary keys, foreign keys, data types, and constraints

L2.2 – Database Schema

The database portableindoorfeedback (also known during development as repif1_leandro) was implemented in MariaDB with the following tables:

Table	Description
consumer	Platform users with username, full name, email, and admin flag
station	Sensor stations linked to an owner and a creator (consumer)
measurement	Environmental readings: temperature, humidity, pressure, light, gas, timestamp
collection	Named groups of measurements created by a user
collects	Join table: which measurements belong to which collection
shares	Join table: which collections are shared with which users
consumer_friend	Friendship pairs between consumers

L2.3 – MariaDB Installation

MariaDB was installed on the Raspberry Pi to host the station's local database:

```
sudo apt update && sudo apt upgrade
sudo apt install mariadb-server -y
sudo mysql
```

If MariaDB [(none)]> appears, the installation is successful.

ATELIER — PCB HOLDER

Printed Circuit Board Assembly

PCB Assembly Overview

The custom PIF 25/26 PCB was received along with all required components. The goal was to correctly solder and assemble the full board for integration into the PIF station enclosure.

Step 1 – Component Inventory

The following components were received alongside the PCB and visually inspected to ensure no damage:

- Pin headers (male and female)
- Screw terminals
- Resistors
- Connectors
- Switch
- Other small electronic components required for the board

Step 2 – Component Placement

Each component was identified using the labels and markings printed on the PCB. Components were placed in their corresponding positions, with careful attention to correct orientation where required (e.g. polarised capacitors, ICs).

Step 3 – Soldering

Once all components were correctly positioned, soldering was performed at the soldering station. The following quality checks were applied:

- Each pin was properly soldered with sufficient solder flow
- All solder joints were clean, solid, and shiny (no cold joints)
- No short circuits were created between adjacent pins

Step 4 – Final Inspection

After soldering, a visual inspection confirmed that all components were securely attached and correctly soldered. The completed PCB was ready for integration into the station assembly.

ATELIER — PVC HOLDER

Mechanical Bracket Fabrication

Overview

Two PVC plastic boards were provided by the school — one thin and one thick. Measurements were taken and holes were drilled through both boards so they could be screwed together to form a structural bracket for the Raspberry Pi and display.

PVC Board 1 (Front/Display Support)

The first board serves as the support surface for the display and Raspberry Pi. The following work was performed:

10. Measurements were marked on the board for all mounting holes and the central cable pass-through.
11. Screw holes were drilled at the required positions.
12. A large rectangular hole was cut in the centre using a hacksaw to allow all cables to pass through.
13. The edges of the cut-out were smoothed using files to achieve a clean, safe finish.
14. Three holes were drilled along the bottom edge to allow screwing to PVC Board 2.

PVC Board 2 (Rear/Structural Board)

The second board required additional work compared to the first, as holes also had to be drilled into the side edges:

15. Measurements were marked on both the main face and the side edges of the board.
16. Side holes were drilled to align with PVC Board 1 for the side attachment screws.
17. Main body holes were drilled for component mounting.
18. The two boards were then aligned and screwed together to form the complete PVC bracket assembly.

The completed PVC bracket was used to mount the Raspberry Pi and display as a single removable unit inside the wooden housing, allowing easy access for maintenance.

A3 — POWER SUPPLY EXTENDER

Cable Preparation & Soldering

Introduction

The objective of this activity was to build a power supply extender cable. The extender allows a power cable to be extended while remaining safe and fully functional. A soldering station was used to join the wires, and heat-shrink tubing was applied for insulation.

Materials & Tools

Category	Items
Materials	Power cable, Heat-shrink tubes, Insulating sleeves
Tools	Soldering station, Solder wire, Wire stripper, Side cutters, Heat gun / hot air source

Procedure

Step 1 – Cable Preparation

The power cable was cut into two sections. The outer insulation was carefully removed from both ends to expose the internal wires.

Step 2 – Wire Identification

The individual wires inside the cable were identified by colour:

- Live (brown)
- Neutral (blue)
- Ground / Earth (yellow-green)

Step 3 – Wire Stripping

The insulation at the tip of each wire was removed using the wire stripper to expose the copper conductors. Heat-shrink tubes were slid onto the wires before soldering to prepare for the insulation step.

Step 4 – Soldering

Matching wires were soldered together at the soldering station:

- Live → Live
- Neutral → Neutral

- Ground → Ground

This produced a strong, reliable electrical connection at each junction.

Step 5 – Insulation

Heat-shrink tubing was positioned over each soldered joint and heat was applied using a heat gun to shrink the tubing tightly around the connection, fully covering all exposed conductors.

Step 6 – Final Assembly

The cable shielding and outer insulation were restored around the joint area to protect the wires and make the extended cable safe for use.

Safety Measures

- The soldering station was handled with care at all times
- All wire junctions were fully insulated — no exposed copper was left
- Correct wire colours were matched to prevent dangerous cross-connections

Conclusion

A power supply extender cable was successfully built. The soldering station enabled strong, secure connections, and heat-shrink tubing provided proper insulation. This activity improved skills in soldering, cable preparation, and electrical safety.

A4 — SENSOR 1: GAS SENSOR & ADC

ADS1115 Integration with Raspberry Pi

Development Overview

Sensor 1 integrates an analog gas sensor (MQ-135, metal can type) with a 16-bit ADS1115 analog-to-digital converter (ADC). This combination provides significantly higher measurement resolution than the Raspberry Pi's native digital inputs. The ADS1115 was selected for its high resolution and I2C communication protocol, which simplifies wiring and reduces GPIO pin usage.

Step 1 – Preparing and Soldering the ADC Module

A male pin header was soldered onto the ADS1115 breakout board. Quality checks:

- Pins are perfectly vertical — no tilt
- No solder bridges between adjacent pads
- I2C pins (VDD, GND, SDA, SCL) and analog inputs (A0–A3) remain clearly accessible

This step is critical to guarantee reliable electrical contact when the ADC is connected to the custom PCB.

Step 2 – Gas Sensor Wiring & PCB Integration

The gas sensor module was mounted on the PCB near the enclosure opening to allow direct airflow exposure:

- Power pins connected to shared +5V and GND rails
- Analog output routed to ADS1115 channel A0
- All connections soldered to ensure mechanical stability and signal integrity

The analog signal path was kept as short as possible to reduce interference.

Step 3 – Power Distribution & Signal Routing

On the custom PCB:

- +5V and GND distributed to both the gas sensor and ADS1115 ADC
- A common ground reference maintained across all components
- The ADC communicates exclusively via I2C (GPIO 2 SDA, GPIO 3 SCL), reducing GPIO usage

Step 4 – Connecting the PCB to the Raspberry Pi

A 40-pin ribbon cable connects the custom PCB to the Raspberry Pi GPIO header:

- 3.3V / 5V — logic and power reference

- GND — common ground
- GPIO 2 (SDA) and GPIO 3 (SCL) — I2C bus

The ribbon cable is routed cleanly through the enclosure cut-out to avoid mechanical stress. This approach allows the sensor board to function like a custom Raspberry Pi HAT.

Step 5 – Mechanical Assembly Inside Enclosure

The PCB and Raspberry Pi are mounted using spacers and screws. The gas sensor aligns with the enclosure opening for proper air exposure. Internal wiring is kept short and secured to prevent vibration or accidental disconnection.

Enclosure Feature	Purpose
Electrical insulation	Prevents short circuits between components
Mechanical protection	Guards against physical damage
Controlled airflow	Ensures accurate sensor readings

Step 6 – Software Configuration & Testing

After hardware assembly:

19. I2C was enabled on the Raspberry Pi (raspi-config → Interfaces → I2C).
20. ADS1115 detected on the I2C bus using: `i2cdetect -y 1`
21. Software was configured to periodically read raw ADC values and convert to voltage using the gain scaling formula.
22. Baseline measurements were recorded in clean air.
23. Sensor response verified under varying air conditions — readings changed accordingly.

Stable and repeatable readings confirmed correct soldering, wiring, and I2C communication.

Conclusion

The gas sensor and 16-bit ADC were successfully integrated and tested. The external ADS1115 converter significantly improved measurement resolution compared to standard analog inputs, enabling accurate and stable data acquisition. The setup provides a solid foundation for further calibration and can be extended to additional sensors or integrated into a larger monitoring system.

ATELIER — FULL STATION ASSEMBLY

Portable Indoor Feedback — Physical Build

Introduction

This section documents the complete physical assembly of the Portable Indoor Feedback station according to the provided work order. The system consists of a Raspberry Pi, sensors, a display, and a control button, all mounted inside a custom wooden housing with a removable cover for easy access and maintenance.

Materials List

Component	Description
Raspberry Pi	Main processing unit
Display (7")	Output screen connected to Raspberry Pi
Wooden housing	4 frame panels + rear panel
PVC bracket	Mount for Raspberry Pi and display
Breadboard	Circuit mounting for sensors
Light sensor (BH1750)	Measures light intensity (Lux)
Barometric sensor (BME280)	Measures atmospheric pressure (hPa) and temperature
Gas sensor (MQ-135)	Measures air quality / gas concentration
Push button	User input (power / interaction)
Jumper wires	Electrical connections between components
USB-C cable	Power supply for Raspberry Pi

Assembly Process

3.1 – Component Check

Before starting assembly, all required components were verified: Raspberry Pi, sensors, breadboard, display, button, and all housing parts.

3.2 – Preparing the Wooden Housing

The wooden housing consists of 5 parts: 4 frames and 1 rear panel. Holes were drilled at precise locations for:

- The push button

- LED cable passthrough
- Sensor openings for airflow

3.3 – Installing the Push Button

The push button was inserted into its drilled hole and secured in place. Wire length was kept sufficient to still allow the display and bracket to be removed for maintenance.

3.4 – LED Openings

Holes for LED cables were drilled in the housing. LEDs were not yet installed at this stage as they had not been received.

3.5 – Mounting the Sensors

The BH1750 light sensor and BME280 barometric sensor were mounted on the rear breadboard, facing outward through the housing, and connected directly to the Raspberry Pi GPIO. They were fixed to the board using screws.

3.6 – Installing Raspberry Pi & Display

The Raspberry Pi and touchscreen display were mounted together onto the PVC bracket, which was then inserted into the wooden housing as a single removable assembly.

3.7 – Connecting All Components

All components were wired together: sensors to GPIO, push button to GPIO, display via ribbon cable, and USB-C power supply connected to the Raspberry Pi.

3.8 – Final Assembly

After verifying all connections, the housing was partially assembled. Internal wiring was arranged neatly before attaching the top cover (partially), keeping the assembly accessible.

System Function

After power-on via the USB-C cable, the Raspberry Pi boots and the sensors collect environmental data. The data is processed and displayed on the touchscreen. The push button provides user interaction (power/reset).

Problems & Solutions

Issue	
	Limited space inside the wooden housing made wire management challenging.

Solution

Wiring was reorganised and excess cable length was reduced to improve routing and prevent interference.

L4 — WEB & DATABASE SERVER

Apache + MariaDB + PHP + phpMyAdmin

Objective

Configure a server (Ubuntu VM) that functions as both a web server and a database server, hosting the PIF PHP website and a MariaDB database, accessible via a web browser on the network.

System Preparation

```
sudo apt update
sudo apt upgrade -y
```

Installing Apache Web Server

```
sudo apt install apache2 -y
systemctl status apache2
sudo systemctl enable apache2
```

Test: open a browser and navigate to <http://localhost> — the Apache default page should appear.
Default web directory: `/var/www/html`

Installing MariaDB

```
sudo apt install mariadb-server mariadb-client -y
systemctl status mariadb
```

Securing the Database

```
sudo mysql_secure_installation
```

Recommended settings: set root password, remove anonymous users (YES), disallow remote root login (YES), remove test database (YES), reload privileges (YES).

Installing PHP

```
sudo apt install php libapache2-mod-php php-mysql php-cli php-curl php-gd php-
mbstring php-xml php-zip -y
php -v
sudo systemctl restart apache2
```

Installing phpMyAdmin

```
sudo apt install phpMyAdmin -y
```

During setup: select apache2 as the web server and configure the database with dbconfig-common.

Configure Apache

```
sudo nano /etc/apache2/conf-available/phpmyadmin.conf
sudo a2enconf phpmyadmin
sudo systemctl restart apache2
```

phpMyAdmin accessible at: <http://localhost/phpmyadmin>

Creating the PIF Database

```
sudo mysql
CREATE DATABASE portableindoorfeedback;
CREATE USER 'pifuser'@'localhost' IDENTIFIED BY '***';
GRANT SELECT, INSERT, UPDATE, DELETE ON portableindoorfeedback.* TO
'pifuser'@'localhost';
FLUSH PRIVILEGES; EXIT;
```

Deploying the Website

The PHP application files were transferred using WinSCP and placed in the web server directory:

```
/var/www/html/pif
sudo chown -R www-data:www-data /var/www/html/pif
sudo chmod -R 755 /var/www/html/pif
```

The website is accessible at <http://localhost/pif> and supports: homepage, user registration, login, dashboard, and database verification.

WEBSITE DEVELOPMENT

PIF Web Platform — Sprint Report

Introduction

The web platform is the core interface between users and the PIF system. It allows users to manage stations, view measurements, organise data into collections, and interact socially with other users through a friends and sharing system.

Technologies Used

Technology	Role
HTML & CSS	Structure and layout
Bootstrap 5	Responsive design framework
PHP	Server-side logic and database interaction
MySQL (MariaDB)	Relational database management
Chart.js	Graphical data visualisation
Apache + PHP	Local server (deployable on Raspberry Pi)

User Management

- User registration and login
- Secure password handling using hashing
- Session-based authentication
- Role-based access control (User and Admin)
- Logout functionality

Access to protected pages is enforced through authentication and authorisation checks.

Stations Management

- Stations are linked to users via ownership
- Users can view and manage their own stations
- Administrators can create, view, and manage all stations
- Access to station data is restricted to owners and administrators
- Each station records environmental measurements: temperature, humidity, pressure, light, gas

Measurements Visualisation

- Displayed in tabular form
- Visualised using charts (Chart.js)
- Filterable by station
- Ordered by timestamp

Collections

- Created by selecting a station and a time range
- Each collection has a name and description
- Measurements automatically linked to the collection
- Users can view collections in a dedicated area

Friends & Sharing System

- Users can send friend requests (accept / reject)
- Collections can be shared only with accepted friends
- Shared collections are read-only
- Removing a friend automatically revokes shared access

Administrator Area

- Manage user accounts — promote or demote administrators
- Manage all stations and all collections
- Safely delete users, stations, or collections

UI & Design

- Dark theme with blue accent colours
- Responsive layout using Bootstrap
- Improved readability and accessibility
- Clear navigation bar for all main sections

Conclusion

The PIF web platform provides a fully functional, secure, and modular environment. Future improvements planned include QR code station registration, advanced dashboards, and API integrations with physical devices.

L5 — DATA TRANSFER

Raspberry Pi to Central Database — Sprint Week 12

Introduction

User Story L5 implements a fully functional data transfer pipeline from IoT sensor stations to the central database. Raspberry Pi-based stations collect environmental measurements and transmit them to the central server via HTTP POST requests handled by a PHP script (send.php).

Requirements

Priority	Requirement
Must	PHP script for data transfer created
Must	Serial number, timestamp, and all measured values transferred and stored correctly
Must	Script accessible on web and database server
Must	Documentation: annotated source code and user guide
Should	PHP script includes a manual data entry form
Should	Form is well-designed and validates input
Could	Script extended to support data querying

Network Configuration

Development environment uses VMware Workstation with two virtual networks:

- VMnet1 — Host-only (192.168.10.0): isolated internal VM communication
- VMnet8 — NAT (192.168.117.0): external connectivity for Raspberry Pi data transmission

The PHP server is reachable at 192.168.117.140 over the VMnet8 NAT interface.

Station Configuration

Each sensor station uses a TOML configuration file at /opt/station/config.toml defining:

- Station serial number
- I2C sensor addresses (MQ-135: 0x48, BME280: 0x76, BH1750: 0x23)
- Measurement interval (seconds)
- Destination server IP and path

PHP Data Transfer Script (send.php)

send.php is a dual-purpose script: it accepts API POST requests from Raspberry Pi stations AND provides a manual web form for operator use.

API Handler

- Detected by HTTP_ACCEPT header containing application/json or api_request=true POST field
- Validates station serial and timestamp
- Checks station existence in database before INSERT
- Inserts measurement using PDO prepared statements (7 parameters)
- Returns JSON response with success/error status

Web Form

- Station dropdown populated from database
- Number inputs for all sensor fields (temperature, humidity, pressure, light, gas)
- HTML5 required validation and placeholder hints
- In-page success/error message on submission

API Reference

Parameter	Description
station_serial	Required — matches database pk_serialNumber
timestamp	Required — format: YYYY-MM-DD HH:MM:SS
temperature	Float — temperature in Celsius
humidity	Float — relative humidity (0–100%)
pressure	Float — atmospheric pressure in hPa
light	Float — light intensity in Lux
gas	Float — gas concentration in ppm

Example curl command:

```
curl -X POST http://192.168.117.140/send.php \  
-d "station_serial=SN-001" -d "timestamp=2024-01-15 14:30:00" \  
-d "temperature=22.5" -d "humidity=45.2" \  
-d "pressure=1013.25" -d "light=150.5" -d "gas=412.3"
```

Testing Results

Test	Result
------	--------

API curl POST	JSON response confirmed new record with assigned ID
Web form submission	In-page success message confirmed, dropdown populated from DB
Missing station serial	Error: Please select a station
Unknown serial via API	JSON error: Station does not exist
Missing required fields	HTML5 validation blocks submission

Conclusion

All Must and Should requirements of User Story L5 were fulfilled. The send.php script successfully bridges Raspberry Pi sensor stations and the central MySQL database. Both automated API transfers and manual operator entries are supported within a single script, with appropriate validation, error handling, and user feedback. PDO prepared statements ensure secure, injection-proof database operations.

BACKUP SERVER

Automated Database Backup & FTP Access

Introduction

This section documents the implementation of a dedicated backup server for the PIF system. The backup server automatically creates daily database backups from the main server and provides secure FTP access to retrieve backup files. Multiple technical challenges were encountered and resolved systematically.

System Architecture

Component	Details
Main Database Server	192.168.0.29 — hosts the PIF MariaDB database
Backup Server	192.168.0.30 — performs automated backups
Raspberry Pi Station	192.168.0.28 — real-time sensor measurements
Backup Storage	/backup/database on the backup server
FTP Access	vsftpd with chroot jail for secure file retrieval

Virtual Machine Setup

Ubuntu Server 24.04 LTS was installed in a VMware VM with:

- Bridged network adapter for LAN communication
- OpenSSH server enabled for remote administration
- User account: cerle025 | Hostname: backup-cerle025

```
sudo apt update && sudo apt upgrade -y
sudo timedatectl set-timezone Europe/Paris
```

Database Remote Access Configuration

MySQL Bind Address

The main database server (192.168.0.29) was reconfigured to accept remote connections:

```
sudo nano /etc/mysql/mysql.conf.d/mysqld.cnf
bind-address = 0.0.0.0
mysqlx-bind-address = 0.0.0.0
sudo systemctl restart mysql
```

Firewall Rule

```
sudo ufw allow 3306/tcp
```

Backup User (Principle of Least Privilege)

```
CREATE USER 'backup_user'@'192.168.0.30' IDENTIFIED BY  
'backup_secure_password_123';  
GRANT SELECT, LOCK TABLES, SHOW VIEW ON portableindoorfeedback.* TO  
'backup_user'@'192.168.0.30';  
GRANT PROCESS ON *.* TO 'backup_user'@'192.168.0.30';  
FLUSH PRIVILEGES;
```

Permission	Purpose
SELECT	Read table contents for backup
LOCK TABLES	Ensure data consistency during backup
SHOW VIEW	Access view definitions
PROCESS	Required for mysqldump operations

Backup Implementation

Secure Credential Storage

```
mysql_config_editor set --login-path=backup --host=192.168.0.29 --  
user=backup_user --password
```

Credentials are stored encrypted in `~/.mylogin.cnf` — no passwords in script files.

Backup Script (`/usr/local/bin/backup_database.sh`)

The bash script performs:

- Creates timestamped MySQL dump files
- Compresses backup files (100MB+ → ~24MB with gzip)
- Logs all operations and errors to `/backup/backup.log`
- Automatically removes backups older than 7 days

```
sudo chmod +x /usr/local/bin/backup_database.sh
```

Automated Scheduling (Cron)

```
0 8 * * * /usr/local/bin/backup_database.sh
```

Backups run automatically every day at 08:00, requiring no manual intervention.

FTP Server Configuration

```
sudo apt install vsftpd -y  
sudo useradd -m -d /backup/database ftpuser  
sudo passwd ftpuser
```

The vsftpd configuration (/etc/vsftpd.conf) jails the FTP user to the backup directory:

```
chroot_local_user=YES
allow_writeable_chroot=YES
user_sub_token=$USER
local_root=/backup/database/$USER
```

This ensures the FTP user can only access /backup/database/ftpuser — a critical security measure.

Database Recovery Procedure

24. Download backup via FTP: ftp 192.168.0.30 → login as ftpuser → get <backup_file>.sql.gz
25. Extract: gunzip <backup_file>.sql.gz
26. Restore: mysql -h 192.168.0.29 -u cerle025 -p portableindoorfeedback < <backup_file>.sql
27. Verify: SELECT COUNT(*) FROM measurement;

Issues & Resolutions

Issue	Solution
MySQL refused remote connections	Changed bind-address to 0.0.0.0 in mysqld.cnf
Firewall blocking port 3306	Added rule: sudo ufw allow 3306/tcp
FTP login failed (chroot dir missing)	Created /backup/database/ftpuser with proper permissions
mysqldump PROCESS privilege warning	Granted PROCESS privilege to backup_user

Conclusion

The backup server is fully configured and operational. The system provides automated daily database backups with encrypted credential storage, secure FTP access for backup retrieval, and comprehensive logging. Security best practices were applied throughout: principle of least privilege for database access, encrypted credential storage, and FTP user chroot jailing. The backup system is ready for production and will provide reliable data protection for the PIF system.

USER STORY L4 — WEB SERVER DETAILED

Step-by-Step Server Configuration Guide

Objective

Install and configure a server that functions as both a web server and database server, hosting a PHP website accessible through a web browser on the network.

Software Stack

- Apache2 — Web server
- MariaDB — Database server
- PHP — Server-side scripting language
- phpMyAdmin — Web interface for database administration

Complete Setup Procedure

1. System Update

```
sudo apt update && sudo apt upgrade -y
```

2. Apache Web Server

```
sudo apt install apache2 -y
systemctl status apache2
sudo systemctl enable apache2
```

Verify: navigate to <http://localhost> in a browser — Apache default page should appear.

3. MariaDB Database Server

```
sudo apt install mariadb-server mariadb-client -y
sudo mysql_secure_installation
```

Secure installation: set root password, remove anonymous users, disallow remote root, remove test DB, reload privileges.

4. PHP Installation

```
sudo apt install php libapache2-mod-php php-mysql php-cli php-curl php-gd php-
mbstring php-xml php-zip -y
php -v
sudo systemctl restart apache2
```

5. phpMyAdmin

```
sudo apt install phpMyAdmin -y
sudo nano /etc/apache2/conf-available/phpmyadmin.conf
sudo a2enconf phpmyadmin && sudo systemctl restart apache2
```

Access: <http://localhost/phpmyadmin>

6. Database Creation

```
sudo mysql
CREATE DATABASE portableindoorfeedback;
CREATE USER 'pifuser'@'localhost' IDENTIFIED BY '***';
GRANT SELECT, INSERT, UPDATE, DELETE ON portableindoorfeedback.* TO
'pifuser'@'localhost';
FLUSH PRIVILEGES; EXIT;
```

7. Website Deployment

```
# Transfer files via WinSCP to /var/www/html/pif
sudo chown -R www-data:www-data /var/www/html/pif
sudo chmod -R 755 /var/www/html/pif
```

Access: <http://localhost/pif>

Final Server Summary

Service	Status
Apache2 (Web Server)	Active — serves PHP application from /var/www/html
MariaDB (Database)	Active — PIF database with dedicated pifuser account
PHP	Installed — version 8.3.6
phpMyAdmin	Active — accessible at /phpmyadmin
PIF Website	Deployed — accessible at /pif

End of Documentation

PIFF — Portable Indoor Feedback | Leandro Cerami | REPIF1 | Lycée Privé Emile Metz | 2025–
2026